

Lecture 12: Apply a CNN to MNIST Handwritten Digit Classification

Yaxin Bi
School of Computing
Ulster University

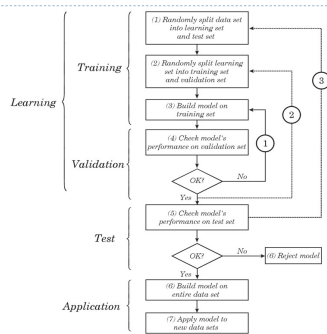
1

Contents

- ▶ K-Folder Cross validation
- ▶ More about Convolution Layers
- ▶ MNIST Handwritten Digit Classification Dataset
- ▶ How to Develop a CNN Model
- ▶ Model Evaluation Methodology
- ▶ How to Develop an Improved Model
- ▶ How to Finalize the Model and Make Predictions

2

Model Evaluation Methodology for ML



3

Example: 2-fold handout

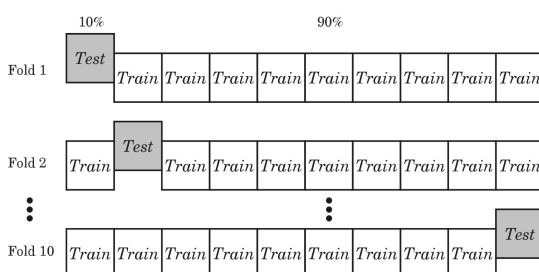
- ▶ Data set contains $n = 4$ cases: 1 2 3 4
- ▶ Perform 2-fold handout estimate
- ▶ Learn on , test on ; number of combinations:



- ▶ Complete 10-fold handout estimate for $n = 100$ cases:
~ 1.73×10^{13} combinations

4

Example: 10-fold Cross Validation



5

Kfold Function from the Scikit-Learn API

```
# example of k-fold cv for a neural net
data = ...
# prepare cross validation
kfold = KFold(5, random_state=1, shuffle=True)
# enumerate splits
for train_ix, test_ix in kfold.split(data):
    model = ...

...
sequence = [i for i in range(20)]
X = np.array(sequence)
kf = KFold(n_splits=2, random_state=None, shuffle=False)
for train, test in kf.split(X):
    print('train: %s, test: %s' % (X[train], X[test]))
```

6

Example: Different Folds

Sequence = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

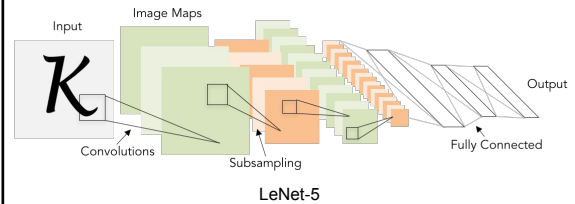
`kf = KFold(n_splits=2, random_state=None, shuffle=False)`
 train: [10 11 12 13 14 15 16 17 18 19], test: [0 1 2 3 4 5 6 7 8 9]
 train: [0 1 2 3 4 5 6 7 8 9], test: [10 11 12 13 14 15 16 17 18 19]

`kf = KFold(n_splits=2, random_state=True, shuffle=True)`
 train: [0 5 8 9 11 12 13 15 18 19], test: [1 2 3 4 6 7 10 14 16 17]
 train: [1 2 3 4 6 7 10 14 16 17], test: [0 5 8 9 11 12 13 15 18 19]

`kf = KFold(n_splits=5, random_state=True, shuffle=True)`
 train: [0 1 2 4 5 7 8 9 11 12 13 14 15 17 18 19], test: [3 6 10 16]
 train: [0 1 3 5 6 7 8 9 10 11 12 13 15 16 18 19], test: [2 4 14 17]
 train: [2 3 4 5 6 8 9 10 11 12 14 15 16 17 18 19], test: [0 1 7 13]
 train: [0 1 2 3 4 5 6 7 8 10 11 12 13 14 16 17], test: [9 15 18 19]
 train: [0 1 2 3 4 6 7 9 10 13 14 15 16 17 18 19], test: [5 8 11 12]

7

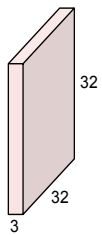
Architecture of LeNet-5



8

Convolution Layer

32x32x3 image



5x5x3 filter

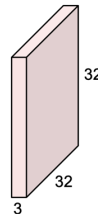


Convolve the filter with the image
 i.e. "slide over the image spatially,
 computing dot products"

9

Convolution Layer (cont'd)

32x32x3 image



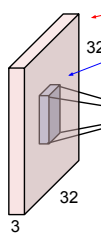
Filters always extend the full
 depth of the input volume

5x5x3 filter

Convolve the filter with the image
 i.e. "slide over the image spatially,
 computing dot products"

10

Convolution Layer (cont'd)

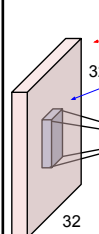


32x32x3 image
 5x5x3 filter w

1 number:
 the result of taking a dot product between the
 filter and a small 5x5x3 chunk of the image
 (i.e. $5 \times 5 \times 3 = 75$ -dimensional dot product + bias)
 $w^T x + b$

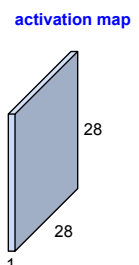
11

Convolution Layer (cont'd)

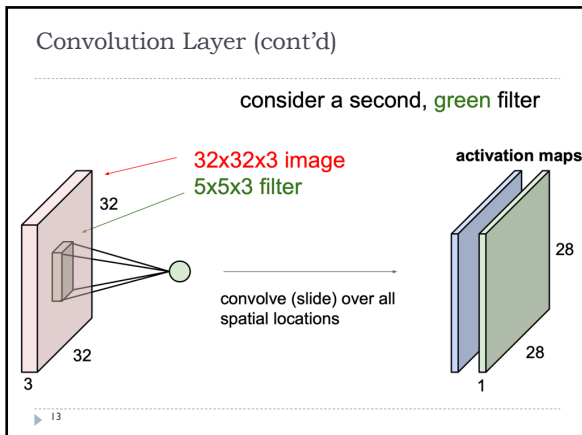


32x32x3 image
 5x5x3 filter

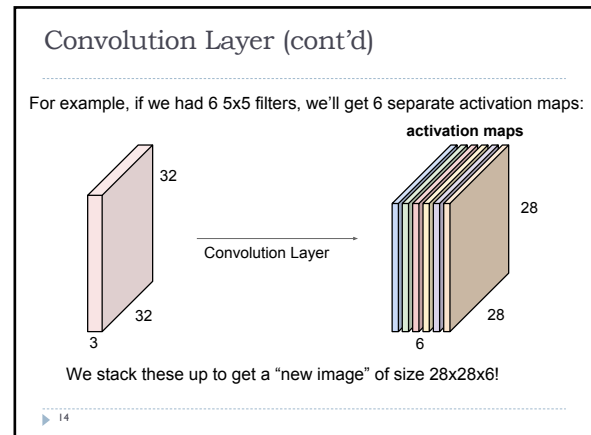
convolve (slide) over all
 spatial locations



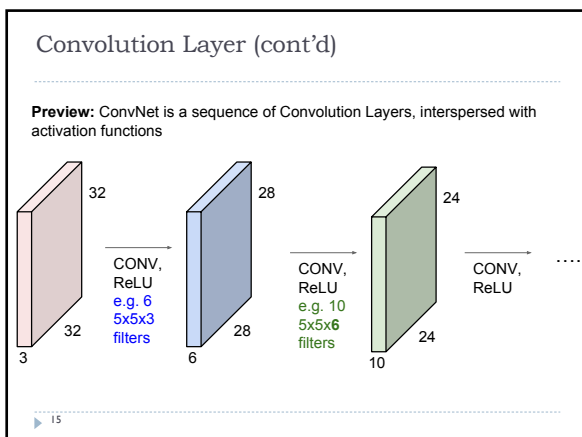
12



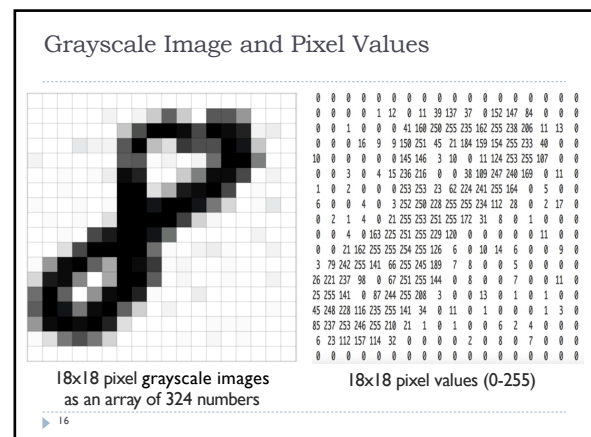
13



14



15



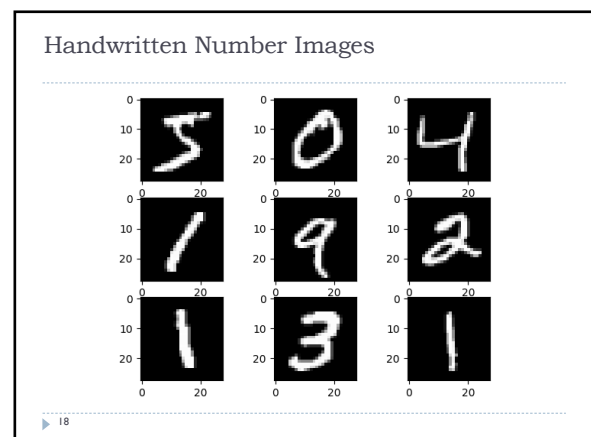
16

MNIST Handwritten Digit Classification

- ▶ The MNIST dataset is an acronym of the **M**odified **N**ational **I**nstitute of **S**tandards and **T**echnology dataset.
 - ▶ it consists of 60,000 small square 28x28 pixel grayscale images
 - ▶ handwritten single digits between 0 and 9.
- ▶ The task is to classify a given image of a handwritten digit into one of 10 classes, representing integers from 0 to 9
- ▶ It is a widely used benchmark dataset. Top-performing models are CNNs that achieved a classification accuracy of above 99%, with an error rate between 0.4 % and 0.2% on the hold out test dataset.
- ▶ Keras has API to get access to the MNIST dataset.

17

17



18

Process for Building a CNN algorithm

1. Reshape the input data into a format suitable for the convolutional layers
2. For class-based classification, one-hot encode the categories
3. Build the CNN model using the Sequential.add() function.
4. Add a pooling layer – not showing all parameters.
5. Add a “flatten” layer which prepares a vector for the fully connected layers
6. Add one or more fully connected layer using Sequential.add(Dense).
7. Follow each fully connected layer with a dropout layer using Sequential.add(Dropout).

19

19

Load Dataset

- ▶ Load the images and reshape the data arrays to have a single color channel

```
(trainX, trainY), (testX, testY) = mnist.load_data()
# reshape dataset to have a single channel
trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
testX = testX.reshape((testX.shape[0], 28, 28, 1))
```

- ▶ There are 10 classes that are represented as unique integers 0 - 9, use a one hot encoding to transform

```
# one hot encode target values
trainY = to_categorical(trainY)
testY = to_categorical(testY)
```

20

20

Rescale the Pixel Values

- ▶ The pixel values for each image in the dataset are unsigned integers in the range between black and white, or 0-255.
- ▶ These values must be scaled prior to providing the images as input to a NN model for training or evaluation
- ▶ Normalize the pixel values of grayscale images, e.g. rescale them to the range [0,1].
- ▶ This involves first converting the data type from unsigned integers to floats, then dividing the pixel values by the maximum value.

```
train_norm = train.astype('float32')
test_norm = test.astype('float32')
train_norm = train_norm / 255.0
test_norm = test_norm / 255.0
```

21

21

Define a CNN Model

- ▶ Define a CNN model for the problem. The model has two main aspects:
 - ▶ front end: the feature extraction comprised of convolutional and pooling layers
 - ▶ backend: the classifier that trains a model and make a prediction.
- ▶ For the convolutional front-end, we can start with a single convolutional layer with a small filter size (3,3) and a modest number of filters (32) followed by a max pooling layer.
- ▶ The activation maps can then be flattened to provide features to the classifier.
- ▶ For a multi-class classification task, an output layer has 10 nodes in order to predict the probability distribution of an image belonging to each of the 10 classes.

22

22

Define a CNN Model (cont'd)

- ▶ The output requires the use of a softmax activation function.
- ▶ Between the feature extractor and the output layer, it requires to add a flatten and a dense layer to interpret the features, in this case with 100 nodes.
- ▶ All layers use the ReLU activation function
- ▶ Hyper parameters setting
 - ▶ the stochastic gradient descent optimizer with a learning rate of 0.01 and a momentum of 0.9.
 - ▶ categorical cross-entropy loss function will be optimized, which is suitable for multi-class classification
 - ▶ the classification accuracy metric, which is appropriate given the same number of examples in each of the 10 classes.
- ▶ Define a model function and return a model

23

23

CNN Model Code

```
def define_model():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu',
        kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
    model.add(MaxPooling2D((2, 2)))
    model.add(Flatten())
    model.add(Dense(100, activation='relu',
        kernel_initializer='he_uniform'))
    model.add(Dense(10, activation='softmax'))
    opt = SGD(lr=0.01, momentum=0.9)
    model.compile(optimizer=opt,
        loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

24

24

Evaluate Model

- ▶ The model will be evaluated using five-fold cross-validation.
 - ▶ k=5 is chosen
 - ▶ each test set is 20% of the dataset, or about 12,000 images
- ▶ The training dataset is shuffled prior to being split, and the sample shuffling is performed each time
- ▶ A modest 10 training epochs with a default batch size of 32 examples
- ▶ The test set for each fold will be used to evaluate the model both during each epoch of the training run, which will be later used to create learning curves
- ▶ Implement an `evaluate_model()` function, taking the training dataset as input and returning a list of accuracy scores and training histories

▶ 25

25

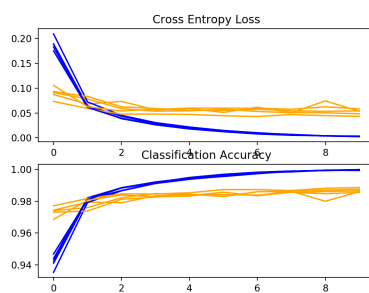
Evaluate Model (cont'd)

```
def evaluate_model(dataX, dataY, n_folds=5):
    scores, histories = list(), list()
    kfold = KFold(n_folds, shuffle=True, random_state=1)
    for train_ix, test_ix in kfold.split(dataX):
        model = define_model()
        trainX, trainY, testX, testY = dataX[train_ix],
        dataY[train_ix], dataX[test_ix], dataY[test_ix]
        history = model.fit(trainX, trainY, epochs=10,
            batch_size=32, validation_data=(testX, testY), verbose=0)
        _, acc = model.evaluate(testX, testY, verbose=0)
        print('> %.3f % (acc * 100.0))
        scores.append(acc)
        histories.append(history)
    return scores, histories
```

▶ 26

26

Evaluation Results



▶ 27

27

Save and Load Model

- ▶ Once the model is evaluated, it can be saved for later use

```
#define a model
model = define_model()
# fit model
model.fit(trainX, trainY, epochs=10, batch_size=32,
    verbose=0)
# save model
model.save('final_model.h5')
```

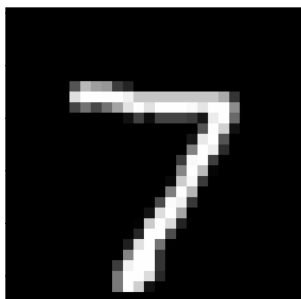
- ▶ The model can be loaded via the `load_model()` function
- ```
model = load_model('final_model.h5')
```

▶ 28

28

## Make Prediction: 7

- ▶ Use the saved model to make a prediction on new images.
- ▶ Assumes a new grayscale image
- ▶ the size of the image is square with 28×28 pixels.
- ▶ Image extracted from the MNIST test dataset.
- ▶ The filename 'sample\_image.png'.
- ▶ Define a function to load the image



▶ 29

29

## Make Prediction: 7

```
def load_image(filename):
 img = load_img(filename, grayscale=True, target_size=(28,
 28))
 img = img_to_array(img)
 img = img.reshape(1, 28, 28, 1)
 img = img.astype('float32')
 img = img / 255.0
 return img

def run_example():
 img = load_image('sample_image.png')
 model = load_model('final_model.h5')
 digit = model.predict_classes(img)
 print(digit[0])
```

▶ 30

30

### Model Improvement: Batch normalization

- ▶ Evaluating the impact that smaller or larger values of the learning rate may have, as well as schedules that change the learning rate during training.
- ▶ Another approach that can rapidly accelerate the learning of a model and can result in large performance improvements is batch normalization.
- ▶ It has the effect of changing the distribution of the output of the layer, specifically by standardizing the outputs, stabilizing and accelerating the learning process.
- ▶ We can update the model definition to use batch normalization after the activation function for the convolutional and dense layers
- ▶ The updated version of `define_model()` function with batch normalization is listed below.

▶ 31

31

### Model Improvement: Batch normalization (cont'd)

```
def define_model():
 model = Sequential()
 model.add(Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
 model.add(BatchNormalization())
 model.add(MaxPooling2D((2, 2)))
 model.add(Flatten())
 model.add(Dense(100, activation='relu',
kernel_initializer='he_uniform'))
 model.add(BatchNormalization())
 model.add(Dense(10, activation='softmax'))
 opt = SGD(lr=0.01, momentum=0.9)
 model.compile(optimizer=opt, loss='categorical_crossentropy',
metrics=['accuracy'])
 return model
```

▶ 32

32

### Model Improvement: Increase in Model Depth

- ▶ There are many ways to change the model configuration in order to explore improvements over the baseline model.
- ▶ Two common approaches involve changing the [capacity](#) of the feature extraction part of the model or changing the capacity or function of the classifier part of the model
  - ▶ the biggest influence is a change to the feature extractor.
- ▶ We can increase the depth of the feature extractor part of the model following a [VGG-like pattern](#)
  - ▶ adding more convolutional and pooling layers with the same sized filter
  - ▶ increasing the number of filters
- ▶ We can add a double convolutional layer with 64 filters each, followed by another max pooling layer.
- ▶ The updated version of the `define_model()` function with this change is listed below.

▶ 33

33

### Model Improvement: Increase in Model Depth (cont'd)

```
def define_model():
 model = Sequential()
 model.add(Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
 model.add(MaxPooling2D((2, 2)))
 model.add(Conv2D(64, (3, 3), activation='relu',
kernel_initializer='he_uniform'))
 model.add(Conv2D(64, (3, 3), activation='relu',
kernel_initializer='he_uniform'))
 model.add(MaxPooling2D((2, 2)))
 model.add(Flatten())
 model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
 model.add(Dense(10, activation='softmax'))
 opt = SGD(lr=0.01, momentum=0.9)
 model.compile(optimizer=opt, loss='categorical_crossentropy',
metrics=['accuracy'])
 return model
```

▶ 34

34